

一阶段review

一、JavaScript 的数据类型与内存模型

1. 原始数据类型 (Primitive Types)

JavaScript 的原始类型有 7 种，特点是值不可变，存储于 **栈内存**。

类型	示例值	特性说明
Number	42, 3.14, NaN, Infinity, -0	所有数字都是浮点数，NaN 为非法数运算结果
String	'abc', "123", hello	不可变字符串，长度属性只读
Boolean	true, false	条件判断结果
Undefined	undefined	未定义变量默认值
Null	null	空对象引用，占位符
Symbol	Symbol('x')	唯一值，用作对象属性名
BigInt	1234567890123456789012345n	用于表示超出 Number 范围的大整数

示例：

```
let a = 10;
let b = a;
b = 20;
console.log(a); // 10 (a 不变, 值拷贝)
```

2. 引用数据类型 (Reference Types)

引用类型值存储在 **堆内存** 中，变量保存的是 **地址引用**。

类型	示例	说明
Object	<code>{ name: 'Tom' }</code>	最基本的复合结构
Array	<code>[1, 2, 3]</code>	特殊对象，拥有 length 和 索引
Function	<code>function() {}</code>	一等公民，可以赋值和传参
Date / RegExp / Map / Set	<code>new Date()</code> , <code>new Set()</code>	特殊内置对象

示例：

```
let obj1 = { name: "Tom" };
let obj2 = obj1;
obj2.name = "Jerry";
console.log(obj1.name); // "Jerry" (共享地址)
```

二、变量声明与提升机制

1. 声明关键字比较

特性	var	let	const
作用域	函数级	块级	块级
提升	是 (undefined)	否 (TDZ)	否 (TDZ)
重复声明	可以	不可以	不可以
赋值	可选	可选	必须初始化

示例：

```
● ● ●
console.log(x); // undefined
var x = 10;

console.log(y); // 报错: Cannot access 'y' before initialization
let y = 10;
```

2. 暂时性死区 (TDZ)

使用 `let` / `const` 声明的变量在其作用域内，在声明前不可访问。

```
● ● ●
{
  // console.log(a); // ReferenceError
  let a = 1;
}
```

三、作用域与作用域链

1. 作用域类型

类型	描述
全局作用域	最外层执行环境（如浏览器中的 window）
函数作用域	函数内部创建的私有作用域
块级作用域	<code>let/const</code> 创建的代码块范围内有效

示例：

```
let a = 1;
function outer() {
  let b = 2;
  function inner() {
    let c = 3;
    console.log(a, b, c); // 可以访问所有父作用域
  }
}
```

2. 作用域链

访问变量时，从当前作用域向上一层查找，直到全局作用域或找不到报错。

```
let x = 1;
function foo() {
  let y = 2;
  function bar() {
    console.log(x + y); // 查找顺序: bar → foo → 全局
  }
}
```

四、执行上下文与变量提升

1. 执行上下文 (Execution Context)

执行 JS 代码时，每一个运行单位（脚本、函数）会创建一个上下文：

阶段	内容
创建阶段	变量提升、作用域链建立、this 绑定
执行阶段	执行代码，赋值、函数调用等

2. 变量提升细节

- **var** 变量会被提前定义，值为 **undefined**
- **function** 函数声明整体提升（包含函数体）
- **let/const** 不提升，进入 TDZ

示例：

```
● ● ●
console.log(foo()); // OK
function foo() {
  return 1;
}

console.log(bar); // undefined
var bar = 2;

console.log(baz); // ReferenceError
let baz = 3;
```

五、值与引用的赋值机制

1. 原始类型：值拷贝

```
let a = 100;  
let b = a;  
b = 200;  
console.log(a); // 100 (值不变)
```

2. 引用类型：地址拷贝

```
let arr1 = [1, 2, 3];  
let arr2 = arr1;  
arr2[0] = 999;  
console.log(arr1[0]); // 999 (共享堆地址)
```

3. 函数参数传递：总是按值，但引用类型值是“地址值”

```
function change(obj) {  
  obj.name = "Changed";  
}  
const person = { name: "Tom" };  
change(person);  
console.log(person.name); // "Changed"
```

六、流程控制结构

1. 条件判断结构

```
if (score >= 90) {  
  console.log("优秀");  
} else if (score >= 60) {  
  console.log("及格");  
} else {  
  console.log("不及格");  
}  
  
switch (color) {  
  case "red": console.log("红色"); break;  
  case "blue": console.log("蓝色"); break;  
  default: console.log("其他");  
}
```

2. 循环结构

类型	示例
for	<code>for (let i = 0; i < 10; i++)</code>
while	<code>while (条件) {}</code>
do...while	<code>do { ... } while(条件);</code>
for...in	遍历对象属性名
for...of	遍历数组值、字符串等可迭代对象

3. 跳出与跳过

- `break`：退出当前循环
- `continue`：跳过当前循环

示例：

```
● ● ●  
for (let i = 1; i <= 5; i++) {  
  if (i === 3) continue;  
  console.log(i); // 跳过 3  
}
```

七、函数与参数

1. 定义方式

```
function add(a, b) {  
  return a + b;  
}  
const sub = function(a, b) {  
  return a - b;  
}  
const mul = (a, b) => a * b;
```

2. 参数与返回值机制

- 所有参数按值传递
- 返回值使用 `return`，无 return 为 `undefined`

示例：

```
function greet(name = "匿名") {  
  return "你好, " + name;  
}
```

八、字符串方法

常用方法：

方法	示例	功能说明
<code>length</code>	<code>'abc'.length</code>	获取长度
<code>charAt()</code>	<code>'abc'.charAt(1)</code> → <code>'b'</code>	获取指定字符
<code>indexOf()</code>	<code>'hello'.indexOf('l')</code> → 2	查找字符位置
<code>slice()</code>	<code>'hello'.slice(1, 4)</code> → <code>'ell'</code>	截取子串
<code>split()</code>	<code>'a,b,c'.split(',')</code> → <code>['a','b','c']</code>	拆分
<code>trim()</code>	<code>' x '.trim()</code> → <code>'x'</code>	去除空格
<code>replace()</code>	<code>'abc'.replace('b', 'x')</code>	替换字符

九、数组方法

增删改查



```
let arr = [1, 2, 3];  
arr.push(4); // [1,2,3,4]  
arr.pop();   // 删除尾部元素  
arr.shift(); // 删除头部  
arr.unshift(0); // 加到头部  
arr.splice(1, 1, 9); // 替换元素
```

遍历与处理



```
arr.forEach((item) => console.log(item));  
let squared = arr.map(x => x * x);  
let even = arr.filter(x => x % 2 === 0);  
let sum = arr.reduce((a, b) => a + b, 0);
```

十、对象基本操作

1. 增删查改



```
let user = { name: "Tom" };  
user.age = 20;  
console.log(user.name);  
delete user.age;
```

2. 遍历对象属性



```
for (let key in user) {  
    console.log(key, user[key]);  
}
```